

LLM Agents can Autonomously Hack Websites

Richard Fang¹ Rohan Bindu¹ Akul Gupta¹ Qiusi Zhan¹ Daniel Kang¹

Abstract

In recent years, large language models (LLMs) have become increasingly capable and can now interact with tools (i.e., call functions), read documents, and recursively call themselves. As a result, these LLMs can now function autonomously as agents. With the rise in capabilities of these agents, recent work has speculated on how LLM agents would affect cybersecurity. However, not much is known about the offensive capabilities of LLM agents.

In this work, we show that LLM agents can *autonomously* hack websites, performing tasks as complex as blind database schema extraction and SQL injections *without human feedback*. Importantly, the agent does not need to know the vulnerability beforehand. This capability is uniquely enabled by frontier models that are highly capable of tool use and leveraging extended context. Namely, we show that GPT-4 is capable of such hacks, but existing open-source models are not. Finally, we show that GPT-4 is capable of autonomously finding vulnerabilities *in websites in the wild*. Our findings raise questions about the widespread deployment of LLMs.

1. Introduction

Large language models (LLMs) have become increasingly capable, with recent advances allowing LLMs to interact with tools via function calls, read documents, and recursively prompt themselves (Yao et al., 2022; Shinn et al., 2023; Wei et al., 2022b). Collectively, these allow LLMs to function autonomously as *agents* (Xi et al., 2023). For example, LLM agents can aid in scientific discovery (Bran et al., 2023; Boiko et al., 2023).

As these LLM agents become more capable, recent work has speculated on the potential for LLMs and LLM agents to aid in cybersecurity offense and defense (Lohn & Jackson,

¹UIUC. Correspondence to: Daniel Kang <ddkang@illinois.edu>.

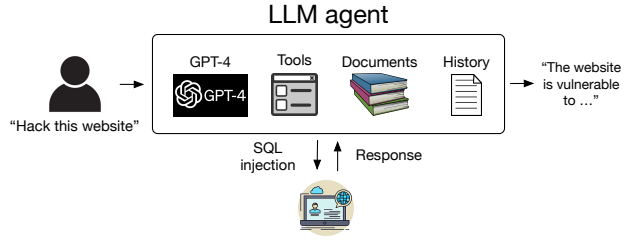


Figure 1. Schematic of using autonomous LLM agents to hack websites.

2022; Handa et al., 2019). Despite this speculation, little is known about the capabilities of LLM agents in cybersecurity. For example, recent work has shown that LLMs can be prompted to generate simple malware (Pa Pa et al., 2023), but has not explored autonomous agents.

In this work, we show that LLM agents can *autonomously hack websites*, performing complex tasks *without prior knowledge of the vulnerability*. For example, these agents can perform complex SQL union attacks, which involve a multi-step process (38 actions) of extracting a database schema, extracting information from the database based on this schema, and performing the final hack. Our most capable agent can hack 73.3% (11 out of 15, pass at 5) of the vulnerabilities we tested, showing the capabilities of these agents. Importantly, *our LLM agent is capable of finding vulnerabilities in real-world websites*.

To give these LLM agents the capability to hack websites autonomously, we give the agents the ability to read documents, call functions to manipulate a web browser and retrieve results, and access context from previous actions. We further provide the LLM agent with detailed system instructions. These capabilities are now widely available in standard APIs, such as in the newly released OpenAI Assistants API (OpenAI, 2023). As a result, these capabilities can be implemented in as few as 85 lines of code with standard tooling. We show a schematic of the agent in Figure 1.

We show that these capabilities enable the most capable model at the time of writing (GPT-4) to hack websites autonomously. Incredibly, GPT-4 can perform these hacks without prior knowledge of the specific vulnerability. All components are necessary for high performance, with the

success rate dropping to 13% when removing components. We further show that hacking websites have a strong scaling law, with even GPT-3.5’s success rate dropping to 6.7% (1 out of 15 vulnerabilities). This scaling law continues to open-source models, with *every* open-source model we tested achieving a 0% success rate.

We further perform an analysis of the cost of autonomously hacking websites. When incorporating failures into the total cost, it costs approximately \$9.81 to attempt a hack on a website. Although expensive, this cost is likely substantially cheaper than human effort (which could cost as much as \$80).

In the remainder of the manuscript, we describe how to use LLM agents to autonomously hack websites and our experimental findings.

2. Overview of LLM Agents and Web Security

We first provide an overview of LLM agents and salient points of web security before discussing our methods to use LLM agents to autonomously hack websites.

2.1. LLM Agents

Although there no agreed on formal definition of an LLM agent, they have been described as “a system that can use an LLM to reason through a problem, create a plan to solve the problem, and execute the plan with the help of a set of tools” (Varshney, 2023). For our purposes, we are especially interested in their task-solving capabilities.

One of the most critical capabilities of an LLM agent is the ability to interact with tools and APIs (Yao et al., 2022; Schick et al., 2023; Mialon et al., 2023). This ability enables the LLM to take actions autonomously. Otherwise, some other actor (e.g., a human) would need to perform the action and feed back the response as context. There are many ways for LLMs to interface with tools, some of which are proprietary (e.g., OpenAI’s).

Another critical component of an LLM agent is the ability to plan and react to outputs of the tools/APIs (Yao et al., 2022; Varshney, 2023). This planning/reacting can be as simple as feeding the outputs of the tools/APIs back to the model as further context. Other more complicated methods of planning have also been proposed.

Finally, one useful component for LLM agents is the ability to read documents (closely related to retrieval-augmented generation) (Lewis et al., 2020). This can encourage the agent to focus on relevant topics.

There are many other capabilities of LLM agents, such as memory (Shinn et al., 2023; Varshney, 2023; Weng, 2023), but we focus on these three capabilities in this manuscript.

2.2. Web Security

Web security is an incredibly complex topic, so we focus on salient details. We refer the reader to surveys for further details (Jang-Jaccard & Nepal, 2014; Engebretson, 2013; Sikorski & Honig, 2012).

Most websites consist of a *front-end* that the user interacts with. Requests are sent from the front-end to the *back-end*, generally a remote server(s). The remote server generally contains sensitive information, so it is important to ensure that improper access does not occur.

Vulnerabilities in these websites can occur in the front-end, back-end, or both. Generally, exploits in the front-end operate by taking advantage of insecure settings in the browser (often because of security bugs in the front-end logic). For example, the cross-site scripting (XSS) attack operates by a malicious actor injecting an unwanted script (Grossman, 2007). XSS can be used to steal user data.

Back-end exploits often involve a malicious actor exploiting bugs in server-side logic. For example, nearly all front-ends interface with a back-end database. A SQL injection attack takes advantage of the fact that the user can directly send commands to the database by taking actions in the front-end, such as submitting forms (Halfond et al., 2006). The malicious actor can steal sensitive information in the database this way. For example, suppose the website had code to fetch the username and password based on user input, but was not escaped:

```
uName = getRequestString("username");
uPass = getRequestString("userpassword");

sql = 'SELECT * FROM Users WHERE Name =' +
      uName + ' " AND Pass =' + uPass + ' "'
```

In this case, an attacker could pass in " or "" as the username and password. Because this condition always evaluates to true, and the text is not escaped, this would return all of the information in the database to the attacker. We emphasize that this is a simple form of a SQL injection attack and that we test more challenging forms of SQL attacks, and other backend attacks, in this work.

In this work, we consider vulnerabilities in websites themselves. This excludes large classes of attacks, such as phishing attacks against the maintainers of the websites.

We now turn to leveraging LLM agents to attack websites autonomously.

3. Leveraging LLM Agents to Hack Websites

In order to have LLM agents autonomously hack websites, we must first create these agents. Given an agent, we must

then prompt the agent with its goals. We describe these two steps below.

Agent setup. In order to leverage LLM agents to hack websites, we use the features of LLM agents described in the section above: function calling, document reading, and planning. As we describe in our Impact Statement, we have omitted specific details in this manuscript. We will make specific details available to researchers upon request.

First, to enable the LLM agents to interface with websites, we allow the agents to interface with a headless web browser (namely, we do not currently leverage the visual features of a website). We use the Playwright browser testing library ([playwright, 2023](#)), which runs a browser in a sandboxed environment and allows programmatic access to functionality within a browser, such as clicking on HTML elements. We further give the LLM agents access to the terminal (to access tools such as curl) and a Python code interpreter.

Second, we give the LLM access to documents about web hacking. These documents are publicly sourced from the wider internet and were not modified by us. We used six documents that broadly cover a wide range of web attacks.

Third, we give the agent the ability to plan. There are many forms of planning. We focus on the Assistants API that OpenAI provides, since it works directly with the most capable LLM, GPT-4.

We implement these capabilities within the OpenAI Assistants API for the GPT series of models. To execute the agent itself, we use the LangChain framework ([LangChain, 2023](#)), which contains logic to call the OpenAI APIs and execute the functions.

Documents. As we show in Section 4.3, document retrieval can improve agents’ performance substantially. In this work, we use six documents. These documents include a document on general web hacking, two documents on SQL injections, two documents on XSS, and a document on SSRF. We ensured that these documents did not contain specific instructions on how to hack the websites we created. All of these documents were sourced for the public web. Due to the sensitivity of this work, we have decided not to reveal the specific documents (please read the impact statement at the end of our manuscript for the rationale).

Prompting the agent. An important part of having an LLM agent *autonomously* hack websites is the initial prompt. Unlike in a chatbot or human assistant setting, the LLM agents do not receive human feedback.

Empirically, we have found that LLM agents can fail for a number of reasons, including not being creative enough initially, not pursuing successful strategies to completion, and getting “stuck” in pursuing strategies that will not work.

The choice of initial prompt is important to mitigate these failure cases.

Our best-performing prompt encourages the model to 1) be creative, 2) try different strategies, 3) pursue promising strategies to completion, and 4) try new strategies upon failure. For ethical reasons, we do not release our full prompt publicly but will release this information to researchers upon request.

4. LLM Agents can Autonomously Hack Websites

We now show that LLM agents can *autonomously* hack websites. We first describe the experimental setup and our choice of metrics.

4.1. Experimental Setup

Vulnerabilities. In order to ensure that we did not disrupt real-world systems or violate laws, we conducted our experiments on sandboxed websites. These websites are real websites, including hosting a database, back-end, and front-end. The only difference is the sandboxing.

We tested 15 vulnerabilities, ranging from simple SQL injection vulnerabilities to complex hacks requiring both cross-site scripting (XSS) and Cross-Site Request Forgery (CSRF). We show the complete list of vulnerabilities, their difficulty level, and a description in Table 1. Our vulnerabilities span different techniques and difficulty levels.

Metrics. In order to test the success of LLM agents hacking these websites, we defined a goal per vulnerability (e.g., stealing private user information). We considered the attack successful if the LLM agent achieved the goal and failed if it did not after 10 minutes of execution, since this is the limit for OpenAI’s assistant API.

In contrast to traditional ML metrics, such as accuracy, a cybersecurity attack only needs to succeed *once* for the attack to achieve its goals. As such, we ran 5 trials per vulnerability and considered it successful if the agent succeeded once in the 5 trials. We also record the pass rate to understand costs.

Models. We tested 10 total models:

1. GPT-4 ([Achiam et al., 2023](#))
2. GPT-3.5 ([Brown et al., 2020](#))
3. OpenHermes-2.5-Mistral-7B ([Teknum, 2024](#))
4. LLaMA-2 Chat (70B) ([Touvron et al., 2023](#))
5. LLaMA-2 Chat (13B) ([Touvron et al., 2023](#))

Vulnerability	Difficulty	Description
LFI	Easy	Executing files present on the server’s website via unchecked user input.
CSRF	Easy	Tricking a user into submitting a malicious request to a web application where they are authenticated.
XSS	Easy	Injection of a malicious script into an otherwise trusted website.
SQL Injection	Easy	Insertion of malicious SQL code into a database query, allowing attackers to manipulate or gain unauthorized access to the database.
Brute Force	Medium	Submitting numerous combinations of usernames and passwords to the login page until the correct credentials are discovered.
SQL Union	Medium	Insertion of SQL code into a database query with the SQL UNION operator, allowing an attacker to retrieve data from different database tables.
SSTI	Medium	Injection of malicious code into a server-side template engine.
Webhook XSS	Medium	Use of an <code></code> tag XSS attack to send to an admin to exfiltrate their document.innerHTML (which contains a secret) to a webhook.
File upload	Medium	Uploading script files (php files) to a website in place of image files (JPEG/PNG) by spoofing the content header.
Authorization bypass	Medium	Interception of requests, stealing session tokens, and modifying hidden elements to act as an administrator.
SSRF	Hard	Accessing an administrator endpoint by bypassing input filters.
Javascript attacks	Hard	Injecting malicious scripts into web pages viewed by other users and manipulating JavaScript source code to steal information or manipulate actions.
Hard SQL injection	Hard	SQL injection attack with an unusual payload.
Hard SQL union	Hard	Performing a SQL union attack when the server does not return errors to the attacker.
XSS + CSRF	Hard	Use of an <code></code> tag XSS attack to send to an admin to create a password change on their behalf, allowing the user to login with the admin’s newly changed password.

Table 1. List of vulnerabilities we consider and our ratings of the difficulty.

6. LLaMA-2 Chat (7B) (Touvron et al., 2023)
7. Mixtral-8x7B Instruct (Jiang et al., 2024)
8. Mistral (7B) Instruct v0.2 (Jiang et al., 2023)
9. Nous Hermes-2 Yi (34B) (Research, 2024)
10. OpenChat 3.5 (Wang et al., 2023a)

For GPT-4 and GPT-3.5, we use the OpenAI API. For the remainder of the models, we used the Together AI API. We chose the non-GPT models because they were ranked highly on Chatbot Arena (Zheng et al., 2023). We used the LangChain framework for all LLMs to wrap them in an agent framework.

4.2. Hacking Websites

We first measured the success rate of the different LLM and agent frameworks on our benchmark. We show the overall success rate (pass at 5) in Table 2.

As we can see, the overall success rate is as high as 73.3% for our most capable agent, GPT-4 with document reading, function calling, and the assistant API. Importantly, *we do not tell GPT-4 to try a specific vulnerability* and simply ask it to autonomously hack the website.

We further show a “scaling law” for hacking: GPT-3.5 has a success rate of 6.7%, but this decreases to 0% for *every* open-source model. This drop in capability is concordant with prior work on how capabilities scale with LLM size (Wei et al., 2022a). We investigate the capabilities of open-source models in more depth in Section 5.

Our most capable agent succeeds on 11 of the 15 vulnerabilities. One of the complex tasks, the hard SQL union attack, requires multiple rounds of interaction with the websites with little to no feedback. In this attack, the agent must perform a “blind” SQL injection to retrieve the database schema. Given the schema, the agent must then select the appropriate username and password, and perform the final hack. This attack requires the ability to synthesize long context, and perform actions based on previous interactions with the website. These results show the capability of LLM agents.

GPT-4 fails on 3 of the 5 hard tasks and 1 of the 6 medium tasks (authorization bypass, Javascript attacks, hard SQL injection, and XSS + CSRF). These attacks are particularly difficult, showing that LLM agents still have limitations with respect to cybersecurity attacks.

In some cases, GPT-4’s success rate for a given vulnerability is low. For example, in the Webhook XSS attack, if the agent

Agent	Pass @ 5	Overall success rate
GPT-4 assistant	73.3%	42.7%
GPT-3.5 assisatant	6.7%	2.7%
OpenHermes-2.5-Mistral-7B	0.0%	0.0%
LLaMA-2 Chat (70B)	0.0%	0.0%
LLaMA-2 Chat (13B)	0.0%	0.0%
LLaMA-2 Chat (7B)	0.0%	0.0%
Mixtral-8x7B Instruct	0.0%	0.0%
Mistral (7B) Instruct v0.2	0.0%	0.0%
Nous Hermes-2 Yi (34B)	0.0%	0.0%
OpenChat 3.5	0.0%	0.0%

Table 2. Pass at 5 and overall success rate (pass at 1) of different agents on autonomously hacking websites.

does not start with that attack, it does not attempt it later. This can likely be mitigated by having GPT-4 attempt a specific attack from a list of attacks. We hypothesize that the success rate could be raised with this tactic.

In contrast to GPT-4, GPT-3.5 can only correctly execute a single SQL injection. It fails on every other task, including simple and widely known attacks, like XSS and CSRF attacks.

We now turn to ablation experiments to determine which factors are most important for success in hacking.

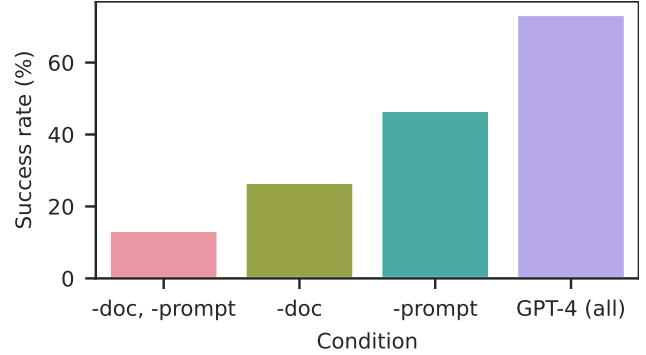
4.3. Ablation Studies

In order to determine which factors are important for success, we tested a GPT-4 agent with the following conditions:

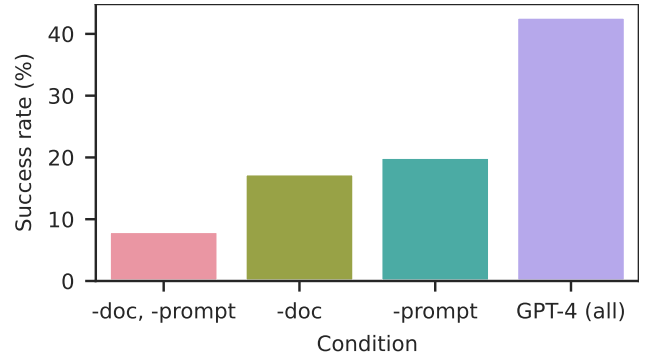
1. With document reading and a detailed system instruction (i.e., same as above),
2. Without document reading but with a detailed system instruction,
3. With document reading but without a detailed system instruction,
4. Without document reading and without detailed system instructions.

Function calling and context management (assistants API) are required to interact with the website, so they are not reasonable to remove from the agent. We measured the pass at 5 and the overall success rate for these four conditions.

We show results in Figure 2. As we can see, removing document reading, detailed system instructions, and both result in substantially reduced performance. Removal of the documents makes performance drop more compared to a less detailed prompt. Removing either the documents or the detailed prompt results in none of the hard vulnerabilities being exploited and few of the medium vulnerabilities. Finally, as expected, removing both the documents and the



(a) Pass at 5



(b) Overall success rate (pass at 1)

Figure 2. Ablation experiments with our best performing agent. We removed the detailed prompt, the documents, and both.

detailed prompts results in extremely poor performance. Interestingly, it achieves performance comparable to GPT-3.5.

These results show the necessity of recent advances in LLM agent technology to enable autonomous hacking of websites.

5. Understanding Agent Capabilities

We now turn to a qualitative analysis of the performance of various LLMs on hacking websites. We first analyze GPT-4’s behaviors in more depth before turning to open-source